

DoomScroller

Technical Design Document

This document explains how the game is built: the main entities (the components that make up the game) and the logic that connects them. It is written so that someone who has not worked on the project can understand how the systems fit together.

Unity project • C# • mobile (portrait) vertical-video game

What is DoomScroller?

DoomScroller is a satirical mobile game that imitates an endless short-video feed (like TikTok or Instagram Reels). The player scrolls an infinite feed of video “reels” and likes them. Engagement keeps a “dopamine” meter topped up; it constantly drains, and if it empties the player loses a life. Occasionally the feed interrupts with a minigame, or inserts a special reel (a reward “holy” reel or a punishing “despair” reel). The goal is to survive and score as high as possible.

How to read this document

The game is built from small, single-purpose components that talk to each other through **events** rather than direct references. Section 2 gives the big picture, Section 3 lists every system grouped by area, and Section 4 walks through the key moment-to-moment flows. A glossary of every script is at the end.

1. The core game loop

Everything in the game revolves around one simple loop. Understanding this loop makes the rest of the systems easy to place:

- **Scroll & like.** The player swipes through an endless feed of video reels and taps to like them.
- **Earn dopamine & score.** Every swipe and every like adds to a dopamine meter and to the score.
- **Fight the drain.** The dopamine meter drains continuously. Engaging with the feed is what keeps it up.
- **Interruptions.** Randomly, a minigame pops up (a fake ad, a time-wasting notification, etc.) or a special reel appears in the feed.
- **Lose a life, or the run.** If the meter empties, the player loses a life and the meter refills. Lose the last life and it is game over.
- **Difficulty ramps.** As the score climbs, the game drains faster, makes top-ups worth less, and spawns minigames more often.

One-line summary

Keep the dopamine meter from emptying by scrolling and liking, survive the minigames and traps, and rack up the highest score before you run out of lives.

2. Architecture overview

The game is a Unity project written in C#. It follows two guiding ideas:

2.1 Small components, one job each

Rather than a few large classes, the game is made of many small **MonoBehaviour** components (scripts attached to objects in the scene). Each one does a single thing — one tracks the dopamine level, one shows the meter bar, one plays the music, and so on. This makes each piece easy to understand and change in isolation.

2.2 Loosely connected through events

Components rarely call each other directly. Instead, when something notable happens, a component **raises an event**, and any interested component **listens** for it. For example, when a reel is liked, the **ReelLike** component fires an `AnyReeLLiked` event; the scoring, dopamine, audio, and combo systems all react to it independently. None of them needs to know about the others.

Why this matters

Because systems communicate through events, you can add, remove, or change a feature (say, a new reaction to liking a reel) without touching the feed or the other systems. It keeps the codebase modular and hard to break.

2.3 The main building blocks

At the centre is the **Feed** object, which hosts the feed controller and most of the gameplay managers. Around it sit display, audio, difficulty, and persistence systems. The diagram below shows the high-level data flow.

Layer	Responsibility
Player input	Swipes and taps on the feed and reels.

Layer	Responsibility
Feed (ReelFeedController)	Spawns/recycles reels, handles scrolling, raises ScrollCommitted / TopReelChanged.
Reels (ReelLike, ReelContent, ReelVideoBackground)	Play the video, show the caption/like UI, raise like events.
Managers on the Feed	Dopamine, scoring, combo, minigames, overdrive, despair, dynamic difficulty — all react to feed events.
Display & audio	Meter bar, vignette, lives, score text, music, SFX — react to manager events.
Persistence & leaderboard	Save the high score locally and sync it to an online leaderboard.

3. Systems & entities

This section groups the components by the area of the game they belong to and explains what each does and how it connects to the rest.

3.1 The feed & reels

The feed is the heart of the game: an endless, scrollable column of full-screen video reels.

Entity	What it does
ReelFeedController	Owens the feed. Spawns a small buffer of reels, moves them 1:1 with the finger, snaps to the nearest reel on release, and recycles reels that scroll off the top so scrolling is infinite. Decides when to insert the special (holy / despair) reels. Raises <code>ScrollCommitted</code> (a reel was advanced) and <code>TopReelChanged</code> (a new reel is on screen).
ReelContent	Per-reel data: the username, caption, audio-name labels, and an optional audio clip. Only the reel currently on top is allowed to play its audio.
ReelVideoBackground	Plays a reel's video full-bleed. Decodes the clip into its own texture and cover-crops it to fill the reel. Also handles muting off-screen reels and a global video-audio volume.
ReelLike	Handles liking a reel (tap the heart or double-tap). Shows the heart pop and particles, and raises the correct event: a normal like, or — for a special reel — the holy or despair event instead.

3.2 Dopamine meter & lives

The dopamine meter is the player's health. It constantly drains and is topped up by engaging with the feed.

Entity	What it does
DopamineManager	The single source of truth for the dopamine level and the player's lives. Adds dopamine on swipes/likes, drains it over time (faster during minigames and as difficulty rises). When it empties, the player loses a life and the meter refills; at zero lives it raises <code>Depleted</code> (game over). Starts on the 2nd of 3 lives. Broadcasts <code>OnDopamineChange</code> , <code>OnLivesChanged</code> and <code>OnLifeLost</code> .
DopamineBar	The on-screen meter. Shows a two-layer sprite animation (background + foreground) that steps from empty to full with the level, with a different frame set per life.
DopamineVignette	A full-screen darkening effect that fades in as the meter runs low and tints redder the fewer lives remain — a “closing in” danger feel.
LivesDisplay	Shows the remaining lives as a sprite (one per life count) and flashes a damage sprite when a life is actually lost.

3.3 Scoring & the like combo

Entity	What it does
ScoreManager	Stores and displays the running score. Deliberately dumb: it only holds the number and updates the label; the rules for awarding points live elsewhere.
FeedScorer	The one place scoring rules live. Listens for swipes, likes and finished minigames and tells the <code>ScoreManager</code> how many points to add. Applies the combo/overdrive multiplier.

Entity	What it does
LikeCombo	Tracks a “like streak”: liking reels in a row builds a combo that, once it passes a threshold, shows a multiplier and raises the score per action. Scrolling past an un-liked normal reel breaks the streak; passing a special reel does not.

3.4 Minigames

Minigames are short interruptions that pop up over the feed. The system is built so new minigames can be added without changing the manager.

Entity	What it does
MinigameManager	Owns the whole minigame system. On each new reel it rolls a chance to pop up a minigame, builds the overlay in code (a dimmed, input-blocking backdrop + a panel that scales in and hosts the chosen minigame), then reports the result. Suppressed during overdrive and on special reels.
IMinigame / MinigameBase	The contract every minigame implements: the manager calls <code>StartGame()</code> and waits for the minigame to raise <code>Finished(Won/Lost)</code> . The manager never needs to know what a specific minigame does.
FakeAdMinigame	A fake ad with a close button that jumps to a new random spot on every tap. Tap it enough times to close it (win).
HeartTapMinigame	Tap a heart to fill it before it drains away; fill it completely to win.
TimeLimitMinigame	A “you’ve been scrolling too long” prompt with look-alike buttons. One triggers game over; the others dismiss it. Buttons are re-shuffled and re-tinted each time so you can’t memorise them.
CallMinigame	An incoming call you dismiss by dragging a button along a winding path without straying off it.
ExampleMinigame	A minimal template minigame (waits a few seconds then finishes) kept as a reference for building new ones.
DynamicDifficulty	Turns the score into a 0–1 difficulty value and pushes it into the other systems: minigames spawn more often and play harder, the meter drains faster, and top-ups are worth less.

3.5 Special reels: holy (overdrive) & despair

Two rare reels are inserted into the feed at random. They share the feed's insertion machinery but trigger opposite outcomes when liked.

Entity	What it does
Overdrive / “holy” reel	A golden reel with its own video, overlay animation and prompt. Liking it starts overdrive : the player jumps to max lives, the score multiplier locks high, minigames stop, custom music and an overlay play, and the meter drains over a fixed timer as the player scrolls for big points. Handled by OverdriveController ; OverdriveScreenShake adds a shake.
Despair reel	The negative counterpart, with its own video, overlay and warning text. Liking it costs a life (or, on the last life, half the current dopamine), plays a sting sound, then auto-scrolls away. Handled by DespairController .

Shared mechanic

Both special reels are tracked the same way: the feed marks the reel, and liking it fires a dedicated event instead of a normal like. Scrolling past either one never breaks the like streak, and minigames never spawn on them.

3.6 Game over

Entity	What it does
GameOverController	Listens for the dopamine system's Depleted event (or a minigame fail). Freezes the game, disables the feed so it can't be scrolled, shows the game-over overlay with the score, reels scrolled and time survived, and saves/shows the high score. The retry button reloads the scene.

3.7 Audio

Entity	What it does
MusicPlayer	Plays the looping main theme. Can be temporarily overridden (e.g. by overdrive music) and then restored.
FeedAudio	Plays the little "dopamine hit" sounds on each swipe and like.
SoundBank	A reusable set of interchangeable sounds with weights, so a repeated action doesn't always play the same clip.

3.8 Saving & the online leaderboard

Progress is stored on the device as JSON files, and the high score is synced to an online leaderboard.

Entity	What it does
SaveSystem	Low-level helper: reads/writes a serializable object to a JSON file in the device's persistent storage.
SaveDataService	Holds the saved data (the high score) and saves it when changed.
SettingsService	Holds settings, including the player's leaderboard username and whether it has been registered online.
LeaderboardService	Talks to the online leaderboard server. On startup it claims a username and uploads the local high score; it can also fetch the top scores. All network requests fail silently so the game works offline.
LeaderboardDisplay / HighscoreCard / MainMenuHighscoreDisplay	UI that shows the leaderboard (a card per entry) and the player's personal best on the main menu.

3.9 Scenes, UI & utilities

Entity	What it does
Scene navigation (MainMenuButtonsSpawner, BackButton, MainMenuGridLayout)	The game is split into scenes (main menu, game, leaderboard, settings, game over). These build the menu buttons and load the matching scenes.
SafeAreaFitter	Keeps UI inside the phone's safe area (away from notches/rounded corners) while the video fills the whole screen.

Entity	What it does
Cheat menu (CheatMenu, CheatState, CheatAccess)	A hidden developer menu (opened by a secret tap, dev builds only) to jump straight into any minigame or a no-drain “scroll mode” for testing.
Effects & helpers (ScrollOverlayAnimation, HeartParticleSpawner, UISpriteAnimation, UIColorFlash, RotateUIImage, FrameRateLimiter, DynamicDifficultyDebug)	Small visual/utility helpers: spritesheet animations, particle bursts, colour flashing, the spinning music disc, a 60 FPS cap to save battery, and an on-screen difficulty debug overlay.

4. Key gameplay flows

These step-by-step walkthroughs show how the components cooperate during the most important moments of play.

4.1 A normal scroll & like

- The player swipes up. **ReelFeedController** moves the reels, recycles the top one, and raises `ScrollCommitted`.
- **DopamineManager** adds a swipe top-up; **FeedScorer** adds swipe points; **FeedAudio** plays a swipe sound; **LikeCombo** updates the streak.
- The player taps like. **ReelLike** raises `AnyReelLiked`.
- **DopamineManager** adds a like top-up, **FeedScorer** adds like points (times the combo multiplier), **LikeCombo** extends the streak, and the heart/particle effects play.
- Meanwhile **DopamineManager** is draining every frame; **DopamineBar** and **DopamineVignette** update to match.

4.2 A minigame interrupts

- A new reel becomes current, raising `TopReelChanged`. **MinigameManager** rolls its spawn chance and wins the roll.
- It builds a dimmed, input-blocking overlay and a panel, spawns the chosen minigame prefab inside, and calls `StartGame()`.
- **DopamineManager** hears the minigame started and drains faster for its duration.
- The player wins or loses; the minigame raises `Finished`. **FeedScorer** awards the result, the overlay scales out, and normal scrolling resumes.

4.3 The holy reel & overdrive

- The feed inserts a holy reel; its overlay and “like now” prompt show while it’s on screen.
- The player likes it → **ReelLike** raises the overdrive event → **OverdriveController** starts the round.
- Lives jump to max, the multiplier locks high, minigames are suppressed, overdrive music and overlay play, the screen shakes, and the meter drains over a fixed timer.
- The player scrolls as much as possible for locked high-multiplier points. When the timer ends, everything reverts: lives back to the start, meter refilled, music and overlay off.

4.4 The despair reel

- The feed inserts a despair reel; its overlay and “dopamine despair” warning show while it’s on screen.
- If the player likes it → **ReelLike** raises the despair event → **DespairController** plays the sting and calls the penalty.
- Not on the last life: lose a life (the meter refills). On the last life: lose half the current dopamine instead, so it can never be an instant game over.
- The feed auto-scrolls to the next reel, ending the despair. Scrolling past it without liking costs nothing and keeps the streak.

4.5 Losing & game over

- The meter hits zero. **DopamineManager** removes a life (`OnLifeLost` → the lives display flashes) and refills the meter, and play continues.
- If that was the last life, it raises `Depleted` instead.

- **GameOverController** freezes the game, disables the feed, shows the final stats, and saves the high score — which **LeaderboardService** later uploads. Retry reloads the scene for a fresh run.

5. Key events (how systems talk)

These are the main signals that connect the systems. “Raised by” is the component that announces the event; “reacted to by” are the components that listen.

Event	Raised by	Reacted to by
ScrollCommitted	ReelFeedController	FeedScorer, DopamineManager, FeedAudio, LikeCombo, ScrollOverlayAnimation
TopReelChanged	ReelFeedController	MinigameManager
AnyReelLiked	ReelLike	FeedScorer, DopamineManager, FeedAudio, LikeCombo
OverdriveLiked	ReelLike	OverdriveController, feed (auto-advance)
DespairLiked	ReelLike	DespairController, feed (auto-advance)
OnDopamineChange	DopamineManager	DopamineBar, DopamineVignette
OnLivesChanged	DopamineManager	DopamineBar, DopamineVignette, LivesDisplay
OnLifeLost	DopamineManager	LivesDisplay (damage flash)
Depleted	DopamineManager	GameOverController
MinigameStarted / MinigameFinished	MinigameManager	DopamineManager, FeedScorer
OnScoreChanged	ScoreManager	DynamicDifficulty, UI

End of document — DoomScroller Technical Design. Generated from the project source for non-team members.